

Filer i C

Filer allmänt

Bakgrund och motivering till att kunna filhantering:

- Många program behöver spara sitt interna tillstånd i permanent minne innan de avslutas, för att de vid någon senare körning ska kunna återta detta tillstånd.
- Datamängden som ska bearbetas är så stor att det inte ryms i minnet (eller är olämplig att ha i minnet).
- Att mellanlagra resultatet av en beräkning på fil, kan vara en bra metod att på ett standardiserat sätt överföra information från ett program till ett annat.

Man använder i C begreppen öppna, accessa och stänga filer. Att "öppna" en fil i C ska inte förväxlas med att "öppna ett dokument" i t.ex. en ordbehandlare. *Hela* sekvensen:

- Öppna en fil i läsläge
- Läs data från fil till internminne
- Stäng filen

motsvarar att "öppna ett dokument". På samma sätt är motsvarigheten till att "spara dokument" ungefär följande:

- Öppna en fil i skrivläge
- Skriv data från internminne till fil
- Stäng filen

För att öppna en fil behövs namnet på filen (dvs. det namn den har i filsystemet, som du kan se i t.ex. utforskaren), och en kod som anger vilket läge filen ska öppnas i (läs-, skrivläge etc). Betydelsen av "öppna" är att systemet förbereder sig för att accessa filen på angivet sätt. Man får ett "kvitto" på att man har öppnat filen. Detta "kvitto" måste man använda då filen sedan ska accessas. Egentligen borde inte så mycket mer behöva sägas om filer, vi behöver bara se hur syntaxen i C är.

"Kvittot"

Det "kvitto" man får från öppningen är en pekare till en fördefinierad typ, **FILE**. Typen är strikt sett inte del av det *språket C*, det är av kategorin egendefinierad typ, men dess definition är redan gjord (den finns i `stdio.h`). Vad denna innehåller vill och skall vi inte veta, vi får absolut inte peta på det som är i den.

Öppna, stänga

Öppningsfunktionen heter

```
FILE *fopen(char *filename, char *mode);
```

Man passar alltså filnamnet, och läget(på strängform) och får tillbaka en pekare till ett FILE-objekt. Exempel:

```
void access_file(char *filename, char *mode)  
{  
    FILE *f;  
    ...  
    f = fopen(filename, mode);  
    ...  
}
```

Förutom att öppna filen för läsning och skrivning kan man vilja öppna den i uppdateringsläge (både läsa och skriva). Man kan också vilja öppna den för att lägga till saker i slutet (och lämna den befintliga delen ograverad).

Man skiljer dessutom mellan access i textläge respektive binärläge (detta går ortogonalt mot de övriga lägena). Innebörden av binärfiler är att vid access sker ingen konvertering, vilket kan ske (i vissa fall) för textfiler. Det finns inga speciella krav på filnamnet för text/binärfiler. Det leder inte heller till att filen får någon speciell "märkning" i filesystemet. Syftet är helt enkelt att i binärfiler lagras godtycklig data, och i textfiler lagras läsbar text (alltså inte tecken mindre än 32, förutom nyrad, carriage return eller tab. speciellt så är 0 inte bra). Konsekvensen är att en textfil kan läsas av editorer, medan vanligen inte innehållet i binärfiler.

- "r" - öppnar för läsning från disk
- "w" - öppnar för skrivning till disk
- "a" - öppnar för tillägg i slutet av filen
- "r+" "w+" resp. "a+" - öppnar för läsning och skrivning
- "rb" "wb" resp. "ab" - öppnar en fil för access i binärläge (kan också kombineras med "+")

Om man begär att öppna en fil för skrivning och en fil med angivet namn redan finns, så kommer denna att tas bort. Om öppnandet misslyckas så returneras en NULL-pekare. Det är många typer av fel som kan leda till att öppnandet misslyckas, några vanliga:

- Filen finns inte, då man öppnar för läsning
- Filnamnet är inte giltigt (plattformsb beroende, om du jobbar med DOS/Windows tänk på att \ blir \\ i strängar)
- Ingen diskett är i, och man har försökt öppna en fil på diskettenheten
- Man refererar till en enhet som inte existerar (DOS/Windows)
- Man har inte accessrätt (olika filesystem har olika system för rättigheter, man kan inte säga något generellt om allt som som kan tänkas inträffa)

När man avslutat arbetet med en fil, ska denna stängas. Detta görs med funktionen

```
fclose(FILE *fp);
```

En typisk kodsekvens i en funktion som arbetar med filer blir då:

```
FILE *f;
f = fopen(filename, mode);
if (f!=NULL) {
    ...
    fclose(f);
}
```

Läs, text

Formaterad läsning görs med

```
fscanf(FILE *fp, char *fmt, ...);
```

Här ska vi alltså använda "kvittot" från öppningen, t.ex.

```
...
FILE *f;
int n;
f=fopen("myfile", "r");
if (f!=NULL)
    fscanf(f, "%d", &n);
    ...
```

Betydelsen är att om filen "myfile" existerar och det först i filen finns t.ex. två bytes med värdena 52 resp. 50 så får n värdet 42.
Läsning av en rad sker med

```
fgets(char *s, int size, FILE *fp);
```

Ex:

```
...
FILE *f;
char string[100];
f=fopen("myfile", "r");
if (f!=NULL)
    fgets(string, 100, f);
...
```

Betydelsen är att om filen "myfile" existerar så kommer string att tilldelas den innehållet i den första raden, inkluderande det avslutande '\n'. Om filen (i textvy) ser ut så här:

```
Kalle
Nisse
Putte
```

Så kommer alltså string att innehålla "Kalle\n".

Skriv, text

Här blir det helt analogt, dvs t.ex.

```
...
FILE *f;
f=fopen("myfile", "w");
if (f!=NULL) {
    fputs("Kalle\n", f);
    fprintf(f, "%d\n", 42);
    fputs("PUTTE\n", f);
    fprintf(f, "%d\n", 13);
    ...
}
```

Skulle ge filen (visad i textvy):

```
Kalle
42
PUTTE
13
```

Skriv och läs, binärt

Här är funktionerna fread och fwrite mer praktiska. Ex:

```
struct t_pers {
    char name[30];
    int shoenummer;
};
...
FILE *f;
struct t_pers pers[10]={{"rut", 42}, {"elis", 13}}, copy[10];
int n=2,m;
f=fopen("myfile", "wb");
```

```

if (f!=NULL) {
    fwrite(&n, sizeof(int), 1, f);
    fwrite(pers, sizeof(struct t_pers), n, f);
    fclose(f);
}
f=fopen("myfile", "rb");
if (f!=NULL) {
    fread(&m, sizeof(int), 1, f);
    fread(copy, sizeof(struct t_pers), m, f);
    fclose(f);
}
...

```

Koden bör vara intuitiv: Först skapar vi en fil, "myfile", i vilken först lagras innehållet i n (2, dvs antalet relevanta objekt i 'pers') och omedelbart därefter innehållet i de två första struct t_pers-objekten som finns i 'pers'. Därefter stängs filen och öppnas för att först läsa ett heltal i från filen, vars betydelse antas vara antalet struct t_pers som följer, och dessa läses därefter till 'copy'.

Av exemplet kan man också lära att man vid filhantering i allmänhet behöver bestämma sig för en layout av filen, ett *protokoll*. Detta måste stämma överens mellan skrivning och läsning. Det finns inga tekniska hinder att t.ex. att istället för att läsa heltalet, direkt börja att läsa struct-objekt, vi måste själva ombesörja att det hanteras rätt.

Ibland används en teknik där man loopar in värden från en fil till dess att man når filslut. Det senare kan detekteras med funktionen

```
int feof(FILE *fp);
```

Den returnerar "sant" om fp lästs till filslut. Alternativt kan man kolla returvärdet från 'fgets' (NULL-pekare om filslut). Observera att filslut indikeras inte förrän man "försökt läsa förbi filslut", vilket brukar leda till kladdiga tester och det kan bli lite olika fall om man har eller inte har nyrad sist i filen och om man läser med 'fgets'. Min rekommendation är därför att anamma tekniken ovan med ett antalsvärde först i filen.

Uppdatera

Att öppna i "+"-läge för att bara uppdatera befintlig fil är nödvändigt t.ex. då filen är för stor för att laddas in i minnet. Vi kan använda

```
fseek(FILE *fp, long movement, int pos);
```

Den flyttar skriv (eller läs) positionen i filen 'movement' bytes framåt från position enligt 'pos'. pos kan vara endera av

- SEEK_SET - räknat från början av filen
- SEEK_CUR - räknat från den punkt i filen man just befinner sig
- SEEK_END - räknat från slutet av filen (movement ska alltså vara negativ här)

Antag exemplet ovan och att elis fötter har växt till sig. Vi vill nu uppdatera filen med det ändrade elis-objektet:

```

...
FILE *f;
pers[i].shoenummer++;
f=fopen("myfile", "wb+");
if (f!=NULL) {
    fseek(f, sizeof(int) + i * sizeof(struct t_pers), SEEK_SET);
    fwrite(pers + i, sizeof(struct t_pers), 1, f);
    fclose(f);
}
...

```

Det avrådes att använda uppdateringsläge med textfiler, annat än mycket speciella fall och då bör man veta vad man gör. Problemet är att t.ex. heltatlsvärdet 42 sparad formaterad till textfil kommer att uppta 2 bytes medan värdet 131313 upptar 6 bytes.